

Attempt 1



In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

▼ Details

COE 311K Final Project - Part 3: Damping and Adaptive Step Sizes

Overview

In Part 3 of the final project, you will enhance your implicit solver from Part 2 by implementing two advanced techniques: **Newton-Raphson damping** and **adaptive step size control**. These methods make your solver more robust and efficient, allowing it to automatically handle difficult regions of the solution while maintaining accuracy.

You will apply these techniques to the **same stiff scalar ODE** you chose in Part 2.

Learning Objectives

By completing this project, you will:

- Understand why Newton-Raphson can fail and how damping prevents divergence
- Implement damped Newton-Raphson for improved robustness
- Learn how adaptive step size control works
- Implement a simple adaptive time-stepping algorithm
- Analyze the performance benefits of adaptive methods
- Compare fixed vs. adaptive step size strategies

What You'll Build

Starting from your Part 2 code (Euler's Backward with basic Newton-Raphson), you will add:

1. **Damped Newton-Raphson** - prevents divergence when initial guess is poor
2. **Adaptive Step Size Control** - automatically adjusts h based on error estimates
3. **Performance Analysis** - quantify the improvements over fixed step size

Part 1: Newton-Raphson Damping

The Problem with Basic Newton-Raphson

Recall the Newton-Raphson update:

$$y^{(k+1)} = y^{(k)} - \frac{g(y^{(k)})}{g'(y^{(k)})}$$

This can fail when:

- Initial guess is far from the solution
- Step size h is very large
- The function g is highly nonlinear in certain regions
- The update overshoots and diverges

The Solution: Damping

Damped Newton-Raphson uses a damping factor $\alpha \in (0, 1]$:

$$y^{(k+1)} = y^{(k)} - \alpha \frac{g(y^{(k)})}{g'(y^{(k)})}$$

where:

- $\alpha = 1$ is full Newton-Raphson (when it's working well)
- $\alpha < 1$ takes smaller, safer steps (when convergence is difficult)

Damping Strategy: Line Search

The idea: Choose α to ensure the residual **decreases** at each iteration.

Algorithm:

1. Compute full Newton step: $\delta = -g(y^{(k)}) / g'(y^{(k)})$
2. Start with $\alpha = 1$ (try full step)
3. While $|g(y^{(k)} + \alpha\delta)| \geq |g(y^{(k)})|$:
 - a. Reduce α (e.g., $\alpha = \alpha/2$)

- b. If α becomes too small, stop (no progress possible)
- 4. Accept: $y^{(k+1)} = y^{(k)} + \alpha \cdot \text{delta}$

Key insight: We only accept steps that **reduce the residual**.

Requirements for Part 3.1

Implement a damped Newton-Raphson solver with:

1. **Line search to choose α**
 - Start with $\alpha = 1$
 - Backtrack (halve α) if residual doesn't decrease
 - Minimum α threshold (e.g., 10^{-4})
2. **Tracking and reporting:**
 - Number of backtracking steps per NR iteration
 - Final α value used
 - Whether damping was needed
3. **Comparison to undamped NR:**
 - Test cases where basic NR fails but damped NR succeeds
 - Demonstrate with very large initial h

Part 2: Adaptive Step Size Control

Why Adaptive Step Sizes?

The problem with fixed step size:

- Solution may change rapidly in some regions, slowly in others
- Fixed h is either:
 - Too large $\rightarrow$ large errors in difficult regions
 - Too small $\rightarrow$ wasting computation in easy regions

Adaptive step size:

- Automatically makes h **small** when solution changes rapidly
- Automatically makes h **large** when solution is smooth
- Maintains target accuracy with minimal computation

The Step-Doubling Approach

The idea is simple: **compare a single big step to two smaller steps**.

Since Euler's Backward is **first-order accurate** ($O(h)$ local error), we can estimate the error by:

Step 1: Take ONE step of size h

$$y_{n+1}^{(h)} = y_n + h \cdot f(t_{n+1}, y_{n+1}^{(h)})$$

$$y_{n+1}(h) = y_n + h \cdot f(t_{n+1}, y_{n+1}(h))$$

Solve this using Newton-Raphson $\rightarrow$ get $y_{n+1}^{(h)}$

Error: $\sim Ch$ for some constant C

Step 2: Take TWO steps of size $h/2$

First half-step: From t_n to $t_n + h/2$

$$y_{n+1/2} = y_n + \frac{h}{2} \cdot f(t_n + h/2, y_{n+1/2})$$

Second half-step: From $t_n + h/2$ to $t_n + h$

$$y_{n+1}^{(h/2)} = y_{n+1/2} + \frac{h}{2} \cdot f(t_n + h, y_{n+1}^{(h/2)})$$

Error for each half-step: $\sim C(h/2) = Ch/2$

Total error for two half-steps: $\sim Ch/2 + Ch/2 = Ch$... wait, same as single step!

BUT: The error constant is different. More importantly, we can use this to **estimate** the error.

Error Estimation Formula

The difference between the two approaches estimates the local error:

$$\text{err} = |y_{n+1}^{(h/2)} - y_{n+1}^{(h)}|$$

Why this works:

- Single step has error $\sim Ch^2$
- Two half-steps have error $\sim Ch^2$ but with better constant
- The difference between them approximates the error in the single step

More rigorous: Using Richardson extrapolation theory, the true solution is approximately:

$$y_{\text{true}} \approx 2y_{n+1}^{(h/2)} - y_{n+1}^{(h)}$$

So the error in the single step is:

$$\text{err} \approx |y_{\text{true}} - y_{n+1}^{(h)}| \approx |y_{n+1}^{(h/2)} - y_{n+1}^{(h)}|$$

Step Size Control Algorithm

Given a **tolerance** `tol`:

1. Take ONE step of size `h` using Euler's Backward
→ get $y_{n+1}^{(h)}$ using Newton-Raphson
2. Take TWO half-steps of size `h/2` using Euler's Backward
→ First: $y_{n+1/2}$ from t_n to $t_n + h/2$
→ Second: $y_{n+1}^{(h/2)}$ from $t_n + h/2$ to $t_n + h$
3. Estimate error:
 $\text{err} = |y_{n+1}^{(h/2)} - y_{n+1}^{(h)}|$
4. Compute new step size:
$$h_{\text{new}} = h * \sqrt{\text{tol} / (\text{err} + \text{eps})}$$

where `eps` = `1e-10` (small number to avoid division by zero)
5. Apply safety factors:
$$h_{\text{new}} = \max(h_{\text{min}}, \min(h_{\text{max}}, \text{safety} * h_{\text{new}}))$$

Typical: `safety` = `0.9`, `h_min` = `1e-6`, `h_max` = `1.0`
6. Decision:
 - If `err < tol`:
 - * ACCEPT the more accurate result: $y_{n+1} = y_{n+1}^{(h/2)}$
 - * Use `h_new` for next step
 - If `err > tol`:
 - * REJECT step
 - * Reduce `h` (use `h_new`) and retry from `t_n`

Key point: When we accept, we use $y_{n+1}^{(h/2)}$ (the two-half-step result) because it's more accurate than $y_{n+1}^{(h)}$!

Cost Analysis

Question: Doesn't taking two half-steps cost more?

Answer: Yes, but it's worth it!

Cost per attempted step:

- Single step of size h : ~ 3 -5 Newton-Raphson iterations
- Two half-steps of size $h/2$: $\sim 2 \times (3-5) = 6$ -10 NR iterations

BUT: With adaptive step size, we can use much larger h when the solution is smooth, saving many steps overall.

Example:

- Fixed $h = 0.01$: need 1000 steps to reach $t = 10$
- Adaptive with $\text{tol} = 1e-4$: might need only 100-200 accepted steps
- Even though each adaptive step costs $2\times$ more, total cost is still less!

Requirements for Part 3.2

Implement adaptive step size control with:

1. Error estimation using step-doubling:

- Take one step of size h
- Take two half-steps of size $h/2$
- Compute error estimate: $\text{err} = |y^{(h/2)} - y^{(h)}|$

2. Step size adjustment:

- Accept/reject logic based on tolerance
- Safety factor (0.9) to prevent overly aggressive changes
- Min/max bounds on h
- Use the more accurate $h/2$ result when accepting

3. Tracking:

- Record h used at each step
- Count accepted vs. rejected steps
- Total Newton-Raphson iterations (this counts the cost!)

4. User controls:

- Target tolerance (e.g., $1e-4$)
- Initial step size h_0
- Min/max step size bounds

Part 3: Combined Implementation

The Complete Adaptive Implicit Solver

Combine both techniques:

For each time step:

1. Try step with current h using damped Newton-Raphson
2. If step succeeds (NR converged):
 - a. Estimate local error
 - b. If error $<$ tolerance:
 - Accept step
 - Compute new h for next step
 - c. If error $>$ tolerance:
 - Reject step
 - Reduce h and retry
3. If NR fails to converge:
 - Reduce h significantly and retry

Implementation Structure

python

```
def adaptive_implicit_euler(f, y0, t0, t_final, h0, tol, ...):
    """
    Adaptive Euler's Backward with damped Newton-Raphson

    Parameters:
    - f: ODE function
    - y0: initial condition
    - t0, t_final: time span
    - h0: initial step size
    - tol: error tolerance
    - h_min, h_max: step size bounds
    - nr_tol: Newton-Raphson tolerance
    - max_nr_iter: max NR iterations

    Returns:
    - t: time points (non-uniform!)
    - y: solution values
    - h_history: step sizes used
    - stats: dictionary with performance statistics
    """

    # Initialize
    t = [t0]
    y = [y0]
    h = h0
    h_history = []

    stats = {
        'accepted_steps': 0,
        'rejected_steps': 0,
        'nr_iterations': [],
        'damping_used': 0,
        'function_evals': 0
    }

    # Main loop
    while t[-1] < t_final:
        # Try a step...
        # (implementation details below)
        pass

    return np.array(t), np.array(y), h_history, stats
```

Deliverable: Jupyter Notebook

Submit a **single Jupyter Notebook** that **extends your Part 2 work** with the following additions:

Section 1: Review of Part 2 (~0.25 pages)

- Brief recap of your problem and basic implicit solver
- State the stiffness parameter and why the problem is stiff

Section 2: Newton-Raphson Damping Implementation (~0.5 pages + code)

- Explanation of damping and line search
- Implementation of damped NR solver
- Demonstration where damping helps (e.g., very large initial h)
- Comparison: basic NR vs. damped NR convergence

Section 3: Adaptive Step Size Implementation (~0.75 pages + code)

- Explanation of step-doubling error estimation
- Implementation with clear comments showing:
 - Single step of size h
 - Two half-steps of size $h/2$
 - Error estimation and step size adjustment
- Visualization of how h adapts during solution

Section 4: Performance Analysis (~0.75 pages + plots)

- **Fixed vs. Adaptive comparison:**
 - Same target accuracy: compare total steps, function evaluations
 - Same step budget: compare final accuracy
- **Plots:**
 - Solution with adaptive time points marked
 - Step size $h(t)$ vs. time
 - Error estimate vs. time
 - Accepted vs. rejected steps
 - Comparison table (fixed vs adaptive)

Section 5: Robustness Testing (~0.5 pages + results)

- Test with very large initial h (e.g., $h_0 = 1.0$)
- Test with very tight tolerance (e.g., $\text{tol} = 10^{-8}$)
- Show that adaptive solver handles both gracefully

Section 6: Conclusions (~0.25 pages)

- When is damping necessary?
- When are adaptive step sizes worth the complexity?

- Key lessons learned
-

Formatting Guidelines

Length

- **Total length:** 6-8 pages when printed
- **Text/markdown:** ~2.5 pages
- **Code:** ~2 pages
- **Plots:** ~2.5 pages (6-8 figures)

Code Requirements

- Well-commented implementation of damped NR
- Well-commented implementation of adaptive stepping
- Modular functions (avoid monolithic code)
- Clear variable names

Plot Requirements

Essential plots:

1. Solution with adaptive time points shown
2. Step size h vs. time
3. Error estimate vs. time
4. Convergence comparison (damped vs. undamped NR)
5. Performance comparison table/bar chart
6. Accept/reject step visualization

All plots must have:

- Descriptive titles
 - Labeled axes with units
 - Legends where appropriate
 - Grid for readability
-

Grading Rubric (25 points)

Newton-Raphson Damping (8 points)

- Correct line search implementation (3 pts)
- Demonstration that damping improves robustness (3 pts)
- Clear comparison to basic NR (2 pts)

Adaptive Step Size Control (8 points)

- Correct error estimation (2 pts)
- Correct accept/reject logic (3 pts)
- Proper step size adjustment with safety factors (3 pts)

Performance Analysis (6 points)

- Comprehensive comparison of fixed vs. adaptive (3 pts)
- Clear performance metrics and plots (3 pts)

Code Quality & Presentation (3 points)

- Well-commented, modular code (1 pts)
- Clear explanations and plots (1 pts)
- Overall organization (1 pts)

Implementation Tips

For Newton-Raphson Damping

python

```
def damped_newton_raphson(y_n, t_next, h, g_func, g_prime_func,
                        tol=1e-8, max_iter=20, alpha_min=1e-4):
    """
    Damped Newton-Raphson with line search
    """
    y_guess = y_n

    for k in range(max_iter):
        g = g_func(y_guess, y_n, t_next, h)

        if abs(g) < tol:
            return y_guess, k, True

        g_prime = g_prime_func(h)
        delta = -g / g_prime

        # Line search for damping factor
        alpha = 1.0
        g_current = abs(g)

        for _ in range(10): # Max 10 backtracking steps
            y_new = y_guess + alpha * delta
            g_new = abs(g_func(y_new, y_n, t_next, h))

            if g_new < g_current: # Sufficient decrease
                break

        alpha = alpha / 2.0

        if alpha < alpha_min:
            print(f"Warning: alpha too small at iter {k}")
            return y_guess, k, False

    y_guess = y_new
```

```
return y_guess, max_iter, False
```

For Error Estimation (Step-Doubling)

python

```
def estimate_error_step_doubling(y_n, t_n, h, nr_solver):
    """
    Estimate error using step-doubling (Richardson extrapolation)

    1. Take ONE step of size h
    2. Take TWO steps of size h/2
    3. Compare the results

    Parameters:
    - y_n: current solution value
    - t_n: current time
    - h: step size to try
    - nr_solver: function that takes one implicit Euler step

    Returns:
    - y_full: result from single step (less accurate)
    - y_half: result from two half-steps (more accurate)
    - error: estimated local error
    """
    # Single step of size h
    y_full, iters_full = nr_solver(y_n, t_n + h, h)

    # Two half-steps of size h/2
    y_half_1, iters_1 = nr_solver(y_n, t_n + h/2, h/2)
    y_half, iters_2 = nr_solver(y_half_1, t_n + h, h/2)

    # Error estimate
    error = abs(y_half - y_full)

    # Total NR iterations for this error estimate
    total_iters = iters_full + iters_1 + iters_2

    return y_full, y_half, error, total_iters


def adaptive_step(y_n, t_n, h, tol, nr_solver,
                  h_min=1e-6, h_max=1.0, safety=0.9):
    """
    Take one adaptive step with error control

    Returns:
    - y_next: accepted solution value
    - h_used: step size that was accepted
    - h_new: suggested step size for next step
    - accepted: True if step was accepted
    - iters: total NR iterations used
    """
    # Try the step with error estimation
    y_full, y_half, error, iters = estimate_error_step_doubling(
        y_n, t_n, h, nr_solver
    )

    # Compute new step size suggestion
    if error < 1e-14: # Error very small
        h_new = min(2.0 * h, h_max)
    else:
        # Standard formula: h_new = h * sqrt(tol/err)
        h_new = h * np.sqrt(tol / error)
        h_new = safety * h_new # Safety factor
        h_new = max(h_min, min(h_max, h_new))

    # Also don't let h change too dramatically
```

```

h_new = max(0.2 * h, min(5.0 * h, h_new))

# Accept or reject?
if error < tol:
    # ACCEPT: use the more accurate two-half-step result
    return y_half, h, h_new, True, iters
else:
    # REJECT: don't update y, suggest smaller h
    return y_n, h, h_new, False, iters

```

For Step Size Control

python

```

def adjust_step_size(h_current, error, tol, safety=0.9,
                    h_min=1e-6, h_max=1.0):
    """
    Compute new step size based on error estimate

    Uses: h_new = h * sqrt(tol / err)
    """
    if error < 1e-14: # Very small error, increase moderately
        h_new = h_current * 2.0
    else:
        h_new = h_current * np.sqrt(tol / error)

    # Apply safety factor
    h_new = safety * h_new

    # Enforce bounds
    h_new = max(h_min, min(h_max, h_new))

    # Don't change h too aggressively
    h_new = max(0.2 * h_current, min(5.0 * h_current, h_new))

    return h_new

```

Testing Strategy

Test 1: Damping Effectiveness

python

```

# Problem where undamped NR fails
h_large = 2.0 # Very large step size

print("Testing undamped NR with h = 2.0:")
try:
    y_undamped, iters, success = basic_newton_raphson(...)
    print(f"Converged: {success}, iterations: {iters}")
except:
    print("Failed to converge!")

print("\nTesting damped NR with h = 2.0:")
y_damped, iters, success = damped_newton_raphson(...)
print(f"Converged: {success}, iterations: {iters}")

```

Test 2: Adaptive vs. Fixed Step Size

python

```
# Fixed step size (baseline)
t_fixed, y_fixed = euler_backward_fixed(f, y0, t0, t_final, h=0.01)

# Adaptive step size (same target accuracy)
t_adapt, y_adapt, h_hist, stats = adaptive_implicit_euler(
    f, y0, t0, t_final, h0=0.1, tol=1e-4
)

print("\nPerformance Comparison:")
print(f"{'Method':<20} {'Steps':<10} {'Func Evals':<15} {'Max Error':<15}")
print("-" * 60)
print(f"{'Fixed (h=0.01)':<20} {len(t_fixed):<10} {len(t_fixed)*3:<15} ...")
print(f"{'Adaptive':<20} {stats['accepted_steps']:<10} {stats['function_evals']:<15} ...")
```

Test 3: Adaptation to Problem Features

python

```
# Visualize how step size adapts
plt.figure(figsize=(12, 8))

plt.subplot(3, 1, 1)
plt.plot(t_adapt, y_adapt, 'b-o', markersize=3)
plt.ylabel('y(t)')
plt.title('Solution with Adaptive Time Points')
plt.grid(True)

plt.subplot(3, 1, 2)
plt.semilogy(t_adapt[:-1], h_hist, 'r-o', markersize=3)
plt.ylabel('Step size h')
plt.title('Adaptive Step Size History')
plt.grid(True)

plt.subplot(3, 1, 3)
# Plot error estimate
plt.semilogy(t_adapt[1:], error_history, 'g-o', markersize=3)
plt.axhline(y=tol, color='k', linestyle='--', label='Tolerance')
plt.ylabel('Error Estimate')
plt.xlabel('t')
plt.title('Error Estimate vs. Time')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

Common Pitfalls to Avoid

Damping Issues:

1. **Infinite loop in line search** - set max backtracking iterations
2. **α too restrictive** - don't require error to decrease by large factor, just any decrease
3. **Not tracking damping** - count when $\alpha < 1$ to show damping helps

Adaptive Step Size Issues:

1. **Counting iterations correctly** - count all NR iterations from both full and half-steps
2. **Step size thrashing** - use safety factors and limit how much h can change
3. **Rejecting too many steps** - tolerance may be too tight, or initial h too large

4. **Not enforcing $h_{\min}/h_{\max}$** - can lead to extremely small or large steps
5. **Using wrong result** - remember to use $y^{(h/2)}$ when accepting, not $y^{(h)}$!

Performance Analysis Issues:

1. **Unfair comparison** - must compare at same accuracy level
 2. **Not counting all work** - track total function evaluations, not just steps
 3. **Cherry-picking results** - test multiple tolerances to show general trends
-

Verification Checklist

Before submission, verify:

- Damped NR converges when basic NR fails (show at least one example)
 - Step size adapts correctly (small h when error is large, large h when error is small)
 - Acceptance rate is reasonable (>70% accepted is good)
 - Adaptive method uses fewer function evaluations than fixed step for same accuracy
 - All plots are clearly labeled and support your analysis
 - Code runs without errors when kernel is restarted
 - Explanations connect the code to the theory
-

Bonus Challenges (Optional, +5 points each, max +10)

Bonus 1: PI Controller for Step Size

Instead of simple proportional control, implement a PI (Proportional-Integral) controller:

$$h_{n+1} = h_n \left(\frac{\text{tol}}{\text{err}_n} \right)^{k_P} \left(\frac{\text{err}_{n-1}}{\text{err}_n} \right)^{k_I}$$

with $k_P = 0.7$, $k_I = 0.4$ (typical values)

Bonus 2: Embedded RK2 Method

Instead of step-doubling with Euler, implement an embedded second-order Runge-Kutta method for error estimation and compare the efficiency.

Bonus 3: Work-Precision Diagram

Create a log-log plot of:

- X-axis: Total work (function evaluations)
- Y-axis: Final error
- Multiple curves for different tolerances
- Compare fixed vs. adaptive strategies

Resources

Python Tips

python

```
# Efficient way to store variable-length arrays
t_list = [] # Use list during computation
y_list = []

# Convert to numpy at the end
t = np.array(t_list)
y = np.array(y_list)

# Use enumerate to track indices
for idx, ti in enumerate(t[:-1]):
    # Can reference current and previous values
    pass
```

Submission Instructions

- **Due Date:** April 30, 2026
- **Submit via Canvas:** Upload your .pdf file
- **File naming:** `LastName_FirstName_COE311K_Part3.pdf`
- **Size limit:** Keep notebook under 10 MB (don't embed huge datasets)
- **Test before submitting:** Restart kernel and run all cells

Connection to Parts 1 & 2

Three-part progression:

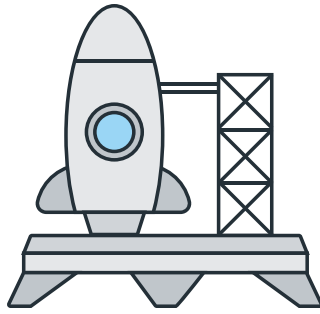
Part	Focus	Methods	Key Insight
Part 1	Non-stiff ODEs	Euler Forward, RK4	Accuracy matters, stability is easy
Part 2	Stiff ODEs	Euler Backward + NR	Stability matters, implicit required
Part 3	Robust & Efficient	Damping + Adaptive	Automation and robustness matter

You're building a production-quality ODE solver!

Choose a submission type

Upload

More



Drag a file here, or

[Choose a file to upload](#)

or



Webcam Photo



Canvas Files

Submit Assignment